BÀI GIẢNG — Từ CV thô đến JSON có cấu trúc: Trích xuất kỹ năng bằng TF-IDF

Môn học: MAT1206E — Nhập môn Trí tuệ Nhân tạo (VNU-HUS, hè 2026) **Đề tài nhóm:** CV/Resume Parser and Job Matching System **Phần tài liệu này giải thích:** chuẩn bị dữ liệu + trích xuất kỹ năng từ CV & JD (tác giả: Đặng Ngọc Quyền)

> **Cách dùng tài liệu này.** Viết cho người **chưa biết gì** về xử lý ngôn ngữ. Mỗi thuật ngữ mới được giải thích ngay khi xuất hiện (in *nghiêng*). Mọi con số trong bài đều là số **chạy thật** từ project, không bịa. Đọc tuần tự chương 0 → 11; hoặc tra nhanh qua mục lục + bảng thuật ngữ (chương 10).

Mục lục

- **Chương 0** — Ai cần đọc, cần chuẩn bị gì
- **Chương 1** — Bài toán tổng thể, và vì sao cần JSON
- **Chương 2** — Dữ liệu gốc: từ 80.017 dòng xuống 994 văn bản
- **Chương 3** — Danh bạ kỹ năng: dạy máy "từ nào là kỹ năng"
- **Chương 4** — TF-IDF: chấm điểm từ thông minh hơn đếm trần
- **Chương 5** — Từ điểm số → 25 kỹ năng cuối
- **Chương 6** — Các trường còn lại của JSON (tên, kinh nghiệm, học vấn, JD)
- **Chương 7** — Chấm điểm máy: precision / recall và bài thi 80/20
- **Chương 8** — Thi ngoài: bộ test 1.099 file từ nguồn khác
- **Chương 9** — Sơ đồ tổng thể + bàn giao cho bạn cùng nhóm
- **Chương 10** — Bảng thuật ngữ
- **Chương 11** — Câu hỏi ôn tập

Chương 0 — Ai cần đọc, cần chuẩn bị gì

Ai cần đọc. Bạn cùng nhóm làm các phần còn lại (matching model, parser/NER, báo cáo) muốn hiểu phần data preparation + skill extraction hoạt động ra sao và **dùng output thế nào**. Hoặc bất kỳ ai muốn học cách xây một pipeline trích xuất thông tin nhỏ hoàn chỉnh.

Cần biết trước. Chỉ cần biết chạy Python. Không cần biết machine learning, không cần biết NLP.

Môi trường của project.

Thứ	Giá trị
Python	3.13
Thư viện chính	`pandas` (bảng dữ liệu), `scikit-learn` (TF-IDF), `pypdf` (đọc PDF)
Chạy script	từ thư mục gốc project: `python data/<tên script>.py`
Dữ liệu gốc	file CSV tải từ HuggingFace (một "chợ dữ liệu" trực tuyến)

Chương 1 — Bài toán tổng thể, và vì sao cần JSON

1.1 Bài toán của cả nhóm

Đề tài: **so CV với tin tuyển dụng (JD = Job Description) để chấm mức độ khớp** — "ứng viên này có phù hợp vị trí Java Developer không?".

Cả nhóm chia làm 3 mảnh. Phần của chúng ta là **đầu vào**: biến đống văn bản thô (CV là đoạn text lộn xộn, JD là đoạn text lộn xộn) thành dữ liệu **sạch, có cấu trúc**, để các phần sau (chấm điểm khớp) làm việc được.

1.2 Vì sao cần JSON

Văn bản thô là một chuỗi chữ không máy nào tính được. JSON (*một định dạng dữ liệu có cấu trúc, kiểu từ điển có khóa → giá trị*) biến mỗi CV thành "phiếu thông tin" có ô rõ ràng:

```
{
  "doc_id": "c94f1a9af5c3",
  "name": null,
  "skills": ["peachtree", "cpa", "accounting"],
  "experience_years": 10,
  "education": {"degree": "B.S", "major": null, "university": "University of Cincinnati"}
}
```

Có cấu trúc rồi thì: - **Matching model** (bạn nhóm) so trực tiếp `skills` của CV với `required_skills` của JD; - **Báo cáo** đếm thống kê được ("62% CV ghi rõ chuyên ngành"); - **Người đọc** nhìn 5 giây hiểu CV nói gì.

1.3 Sơ đồ 30 giây

```
CSV thô 80.017+13.716 dòng
        │  build_pairs.py  (làm sạch, dedup, ẩn thông tin cá nhân)
        ▼
994 văn bản sạch (643 CV + 351 JD)
        │  extract_json.py  (TF-IDF + regex)
        ▼
data/extracted/{cv,jd}/<doc_id>.json   ← sản phẩm bàn giao
```

Chương 2 — Dữ liệu gốc: từ 80.017 dòng xuống 994 văn bản

2.1 Phát hiện đầu tiên: dữ liệu lặp rất nhiều

File `matching_train.csv` nặng **698 MB**, có **80.017 dòng** — mỗi dòng là một cặp (CV, JD, nhãn). Nhưng khi đếm văn bản *khác nhau*, chỉ còn **642 CV** và **280 JD**. Nghĩa là cùng một CV xuất hiện hàng trăm lần, ghép với nhiều JD khác nhau. File test thêm chút nữa: tổng cộng **643 CV + 351 JD = 994 văn bản duy nhất**.

Bài học: luôn đếm "số phần tử thật" trước khi tin con số to đẹp. 80.017 dòng nghe lớn, nhưng tri thức thật trong đó chỉ bằng 994 văn bản.

2.2 Ba khái niệm xử lý cơ bản

1. Dedup (loại trùng lặp). Giữ mỗi văn bản duy nhất một lần. Cách nhận diện "giống nhau" không phải bằng tên file mà bằng **nội dung**: băm (*hash* — hàm biến văn bản thành một dãy ký tự nhận dạng, hai văn bản giống hệt nhau thì băm ra giống hệt nhau) văn bản bằng **md5**, lấy 12 ký tự đầu làm `doc_id` (định danh tài liệu):

```
def doc_id(text: str) -> str:
    """Định danh tài liệu bằng md5 của toàn bộ văn bản (12 ký tự đầu)."""
    return hashlib.md5(text.encode("utf-8")).hexdigest()[:12]
```

Ví dụ: CV kế toán ở chương 6 có `doc_id = "c94f1a9af5c3"` — đó chính là 12 ký tự md5 đầu của nó. Từ giờ muốn nhắc tới CV nào chỉ cần 12 ký tự này, và tên file JSON cũng là nó.

2. PII masking (che thông tin cá nhân). PII = *Personally Identifiable Information* — email, số điện thoại, link cá nhân. Không cần thiết cho việc trích kỹ năng, lại nhạy cảm, nên thay bằng nhãn vô hại:

```
def mask_pii(text: str) -> str:
    for pattern, repl in _PII_RE:      # list các (regex, nhãn thay thế)
        text = pattern.sub(repl, text) # regex = "công thức tìm chữ", sub = thay thế
    return text
```

`_PII_RE` gồm 3 quy tắc: email → `[EMAIL]`, dãy số kiểu điện thoại → `[PHONE]`, đường link → `[URL]`. Ví dụ: `"liên hệ john@mail.com"` → `"liên hệ [EMAIL]"`.

3. Bảng cặp (pairs.csv). Sau làm sạch, lưu 2 thứ: (a) bảng cặp 93.733 dòng — `pair_id, resume_id, jd_id, label, split, domain` — không chứa văn bản, chỉ chứa *khóa nối*; (b) bảng văn bản đã sạch `cv_docs.csv` (643 dòng) và `jd_docs.csv` (351 dòng), mỗi dòng một `doc_id` + toàn bộ chữ. Tách cặp khỏi văn bản để bạn nhóm dễ tra.

Nhãn (label) trong pairs.csv là mức khớp do người gán: `No Fit` 60.313 cặp / `Potential` 13.907 / `Good` 5.797 — lệch nặng về "không khớp", thông tin đáng nhớ khi bạn nhóm huấn luyện model (chương 9).

2.3 Ví dụ chạy thật

```
80.017 dòng train + 13.716 dòng test
  → 93.733 cặp unique
  → 643 CV unique + 351 JD unique
  → 994 văn bản đã ẩn PII, có doc_id
```

Chương 3 — Danh bạ kỹ năng: dạy máy "từ nào là kỹ năng"

3.1 Vấn đề

TF-IDF (chương 4) chấm điểm **tất cả** từ: "management", "responsible", "team"... đều được điểm, kể cả khi chúng không phải kỹ năng. Cần một **danh bạ (lexicon)** — danh sách các từ/ cụm từ được công nhận là kỹ năng — để lọc.

3.2 Hai lớp danh bạ

Lớp 1 — gieo từ dữ liệu thật (707 kỹ năng). Rất nhiều CV có mục "Skills: Java, SQL, ..." ở cuối. Ta tách các mục này ra, đếm xem mục nào xuất hiện ở **ít nhất 5 CV** (từ hiếm hoi xuất hiện 1 lần nhiều khả năng là lỗi gõ) — được 707 kỹ năng tin cậy, lưu ở `data/skills_lexicon.txt` (file văn bản thường, muốn thêm/sửa kỹ năng chỉ cần mở file sửa).

Lớp 2 — nguồn ngoài (6.108 kỹ năng). 707 kỹ năng là *điều máy học từ chính dữ liệu này* — sẽ bị "bóng" của dữ liệu: nếu 994 CV không ai biết Docker thì Docker không vào danh bạ. Nên gom thêm từ 3 bộ dữ liệu công khai trên HuggingFace (job-skill-set, esco/skillspan, techwolf), làm sạch cùng quy tắc (2–40 ký tự, ≤4 từ, không toàn số) → 6.108 kỹ năng.

Danh bạ tổng: 6.815 kỹ năng.

3.3 Code tách block Skills

```
def parse_skills_block(text: str, max_chars: int = 2000, max_items: int = 80):
    """Tách danh sách skill sau từ 'Skills' cuối cùng của CV (pseudo ground truth)."""
    last = None
    for last in SKILL_HDR_RE.finditer(str(text)):
        pass  # lấy lần xuất hiện CUỐI (block Skills thường nằm cuối CV)
    if last is None:
        return []
    tail = str(text)[last.end(): last.end() + max_chars]
    items = []
    for raw in tail.split(",")[:max_items]:
        s = norm(raw.strip(" .;:-"))
        # lọc nhiễu: bỏ quá dài / quá 4 từ / toàn số
        if 2 <= len(s) <= 40 and len(s.split()) <= 4 and not s.isdigit():
            items.append(s)
    return items
```

Giải thích từng dòng: - `SKILL_HDR_RE` là regex tìm đúng chữ "Skills" đứng riêng (không khớp "Skillset"); `finditer` trả về *tất cả* chỗ khớp — vòng lặp `for...pass` chỉ để giữ **chỗ cuối cùng**, vì mục Skills thật thường nằm cuối CV (đầu CV hay có dòng "Skills required: ..."). - `tail`: cắt 2.000 ký tự ngay sau chữ "Skills" — vùng chứa danh sách. - `tail.split(",")`: danh sách kỹ năng thường ngăn cách bởi dấu phẩy → cắt theo phẩy, tối đa 80 mục. - `norm(...)`: viết thường + gọn khoảng trắng (" Java " → "java"). - Dòng lọc: mục ngắn quá (<2 ký tự), dài quá (>40 ký tự hay >4 từ — là câu chứ không phải kỹ năng), hoặc toàn chữ số → bỏ.

Ví dụ. Cho đoạn `"... PROJECT SKILLS: Java, SQL, Spring Boot, 2019, Team player"` → `["java", "sql", "spring boot", "team player"]` ("2019" bị loại vì toàn số).

3.4 Code dựng danh bạ từ nhiều CV

```
def build_lexicon(texts, min_docs: int = LEXICON_MIN_DOCS) -> list[str]:
    """Đếm tần suất item trong block Skills trên NHIỀU doc, giữ item phổ biến."""
    counts: dict[str, int] = {}
    for t in texts:
        for s in set(parse_skills_block(t)):  # set: đếm theo SỐ DOC, không theo số lần
            counts[s] = counts.get(s, 0) + 1
    ok = ((s, c) for s, c in counts.items() if c >= min_docs and s not in STOP_SKILLS)
    return sorted((s for s, _ in ok), key=len, reverse=True)
```

Điểm tinh tế: `set(parse_skills_block(t))` — vì gọi qua `set`, một kỹ năng lặp 10 lần trong *CÙNG MỘT* CV chỉ được đếm 1 lần. Ta muốn "kỹ năng này quen thuộc với bao nhiêu **người**", không phải "CV này thích viết nó mấy lần". `STOP_SKILLS` là 10 từ rác hay gặp (state, client, phone, letters, ...) bị loại thủ công.

Chương 4 — TF-IDF: chấm điểm từ thông minh hơn đếm trần

Đây là trái tim của phần trích xuất. Đọc kỹ chương này là hiểu 80% pipeline.

4.1 Vì sao đếm trần thì dở

Cách ngây thơ nhất: đếm từ nào xuất hiện nhiều trong CV → từ đó quan trọng. Nhưng:

- "experience" xuất hiện trong **798/994** văn bản — đếm trần thì nó top 1 mọi CV, trong khi nó chẳng nói gì về ứng viên này.
- "peachtree" (phần mềm kế toán) chỉ xuất hiện ở **15** CV — nhưng nếu CV *này* có nó, đó là dấu hiệu đặc trưng rất mạnh.

Cần một cách chấm: **từ quan trọng = xuất hiện nhiều trong văn bản này, NHƯNG hiếm ở nơi khác.** Đó chính là TF-IDF.

4.2 Hai thành phần

TF (Term Frequency — tần suất từ trong văn bản này). Đếm số lần từ xuất hiện, rồi *làm dịu bằng logarit* (tham số `sublinear_tf`):

```
tf = 1 + ln(đếm)
```

Logarit tự nhiên `ln` làm số lớn "xẹp" xuống: xuất hiện 10 lần không quan trọng gấp 10 lần mà chỉ gấp ~2,4 lần (vì ln(10) ≈ 2,3). Tránh để từ lặp nhiều quá lấn át.

IDF (Inverse Document Frequency — nghịch độ hiếm giữa các văn bản). Với *n* văn bản:

```
idf = ln((1 + n) / (1 + df)) + 1
```

trong đó `df` = số văn bản chứa từ đó. Từ càng hiếm (df nhỏ) → idf càng lớn. Cộng 1 hai chỗ để không bao giờ ra 0 hay số âm.

Điểm cuối = TF × IDF, rồi **chuẩn hóa L2** (chia toàn bộ vector cho độ dài của chính nó căn bậc hai tổng bình phương — để độ dài vector luôn bằng 1). Mục đích chuẩn hóa: CV dài 5 trang và CV ngắn 1 trang so sánh công bằng, không phải CV nào chữ dày hơn là thắng.

4.3 Bảng số thật — chạy trên CV kế toán c94f1a9af5c3

(Máy fit trên 994 văn bản; từ vựng gồm **32.704** mục từ.)

Từ	Đếm trong CV này	df (số văn bản chứa /994)	TF = 1+ln(đếm)	IDF	TF×IDF	Điểm sau L2
peachtree	2	15	1.69	5.13	8.7	**0.1695**
cpa	3	55	2.10	3.88	8.1	0.1588
accounting	7	310	2.95	2.16	6.4	0.1178
experience	0*	798	—	1.22	—	0.0000

* Kỳ thú: trong CV này có chữ "experienced" — đếm *chuỗi con* (substring — dãy ký tự nằm trong dãy khác, "cat" là chuỗi con của "category") thì "experience" thấy 1 lần. Nhưng bộ tách từ (*tokenizer*) chỉ nhận **từ nguyên vẹn**: "experienced" là một từ khác, không sinh token "experience" → điểm đúng bằng 0. Đây là lý do sau này cần "máy dò" thứ hai (chương 5.2).

Đọc bảng: **"peachtree" chỉ 2 lần vẫn thắng "accounting" 7 lần** (8.7 vs 6.4) — độ hiếm bù quá mức đếm. Còn "experience" có df 798 → idf chỉ 1.22, gần mức sàn. Cơ chế hoạt động đúng như thiết kế.

4.4 Code fit — và giải thích 4 tham số

```
def fit(texts) -> TfidfVectorizer:
    vec = TfidfVectorizer(ngram_range=(1, 2), min_df=3, stop_words="english", sublinear_tf=True)
    vec.fit(texts)
    return vec
```

- `ngram_range=(1, 2)`: ngoài từ đơn còn giữ **bigram** (cặp 2 từ liền nhau) — "sql" và "sql server" là 2 mục từ riêng.
- `min_df=3`: bỏ mục từ xuất hiện trong <3 văn bản (lọc lỗi gõ một lần gặp).
- `stop_words="english"`: bỏ từ vô nghĩa tiếng Anh (the, and, with...).
- `sublinear_tf=True`: bật TF làm dịu logarit như trên.

`fit` = *học từ toàn bộ 994 văn bản*: dựng từ vựng + tính bảng df (để có idf). Đây là bước "soạn giáo án" — chưa đụng tới việc chấm từng văn bản.

4.5 Top-8 thật của CV mẫu — và 2 bài học

Top 8 mục từ điểm cao nhất của CV c94f1a9af5c3:

#	Mục từ	Điểm
1	cpa firm	0.2079
2	peachtree accounting	0.2079
3	area network	0.2019
4	installed new	0.1924
5	local area	0.1885
6	web site	0.1790
7	accountant 01	0.1739
8	cincinnati	0.1739

Hai bài học nhìn thẳng vào:

1. **Bigram hiếm chiếm hết bảng** — "cpa firm", "peachtree accounting" là cặp từ ít ai viết, idf khủng. Điều này tốt cho độ đặc trưng, nhưng nghĩa là **bắt buộc** phải lọc qua danh bạ: "installed new", "web site" không phải kỹ năng.
2. **Rác lọt được**: "accountant 01" là chức danh dính mảnh ngày "01/2013" — bộ tách từ cắt bỏ dấu "/". Không có máy hoàn hảo; pipeline phải **nhiều tầng lọc** (điểm cao → có trong danh bạ → không trùng nhau → top 25).

Chương 5 — Từ điểm số → 25 kỹ năng cuối

5.1 `topk_of` — lấy top điểm, lọc danh bạ, dừng đúng 25

```
def topk_of(X, i, features, lex_set, top_k=TOP_K):
    row = X.getrow(i)
    order = sorted(range(len(row.data)), key=lambda j: (-row.data[j], features[row.indices[j]]))
    out = []
    for j in order[:TOP_CANDIDATES]:
        term = features[row.indices[j]]
        if term in lex_set:
            out.append((term, float(row.data[j])))
            if len(out) == top_k:
                break
    kept = set(dedup_overlap([t for t, _ in out]))
    return [(t, s) for t, s in out if t in kept]
```

Từng dòng: - `X` là *ma trận thưa* (sparse matrix — bảng điểm mà đa số ô là 0, máy chỉ lưu các ô khác 0 cho nhẹ). `getrow(i)` lấy dòng i = điểm của một văn bản. - `order`: sắp thứ tự các ô theo điểm **giảm dần** (`-row.data[j]` vì sorted tăng dần mặc định); khi hai ô bằng điểm, xếp theo tên (`features[...]`) — để hai lần chạy cho kết quả *giống hệt nhau*. - Vòng lặp duyệt **tối đa 200 ô cao nhất** (`TOP_CANDIDATES=200`), chỉ giữ ô mà tên nằm trong danh bạ; **dừng ngay khi đủ 25**. - Cuối cùng lọc trùng bằng `dedup_overlap` (mục 5.3).

Vì sao dừng ở 200 thay vì quét hết 32.704 mục? Tối ưu nhỏ: kỹ năng nằm ngoài top 200 điểm thì cũng chẳng được chọn vào top 25.

5.2 Máy dò thứ hai: scan regex

TF-IDF có 3 điểm mù: 1. **Từ nguyên vẹn** — "experienced" không sinh "experience"; 2. **Từ hiếm bị min_df loại** — một kỹ năng chỉ xuất hiện ở 2/994 văn bản bị vứt khỏi từ vựng; 3. **Danh bạ 6.815 kỹ năng** chỉ dùng để lọc — nhưng có kỹ năng nằm ngoài top 200 điểm dù CV *có viết* nó.

Nên thêm máy dò thứ hai kiểu "Ctrl+F" quét **toàn bộ văn bản** tìm từng kỹ năng trong danh bạ:

```
def make_scan_re(lexicon: list[str]):
    """Regex Ctrl+F: (?<![a-z0-9])(kỹ năng 1|kỹ năng 2|...)(?![a-z0-9]) — dài trước ngắn sau."""
    alts = sorted((re.escape(t) for t in lexicon), key=len, reverse=True)
    return re.compile("(?<![a-z0-9])(?:" + "|".join(alts) + ")(?![a-z0-9])")
```

Giải phẫu cái regex (biểu thức chính quy — "công thức tìm chữ"):

- `re.escape(t)` — *thoát* các ký tự đặc biệt của kỹ năng, để "c++" tìm đúng chữ "c++" chứ không hiểu "+" là toán tử của regex.
- `(...|...)` — hoặc: khớp kỹ năng này HOẶC kỹ năng kia.
- **Sắp dài trước, ngắn sau**: regex với "hoặc" khớp nhánh đầu tiên gặp được — nếu "sql" đứng trước "sql server", chỗ nào có "sql server" regex cũng dừng ở "sql" và bỏ lỡ bản dài.
- `(?<![a-z0-9])` — *lookaround nhìn lùi* (kiểm tra mà không "ăn" ký tự): "ký tự NGAY TRƯỚC không được là chữ cái thường hoặc số".
- `(?![a-z0-9])` — *lookaround nhìn tới*: "ký tự NGAY SAU không được là chữ cái thường hoặc số".

Vì sao không dùng `\b` (biên từ — ranh giới chữ/không-chữ) như người ta vẫn làm? Vì kỹ năng chứa ký tự đặc biệt. Xét "c++": sau dấu `+` là dấu phẩy. `\b` yêu cầu một bên là ký tự từ (chữ/số/gạch dưới) và bên kia không — mà `+` và `,` ĐỀU không phải ký tự từ → không có biên nào ở đó → "c++" **không bao giờ** khớp với `\bc\+\+\b`. Lookaround "không phải chữ/số" thì không quan tâm bên kia là gì → dấu phẩy được thông qua. (Lỗi này từng có thật: sửa xong bắt được thêm 362 dòng kỹ năng chứa ký tự đặc biệt như c++, .net, c#.)

5.3 `dedup_overlap` — bỏ từ bị chứa

Quét xong thường có cả "sql" lẫn "sql server" — nhưng hai ô gần trùng nhau chiếm 2 slot top-25 trong khi đáp án chỉ liệt kê 1 dạng. Nên bỏ từ bị **chứa hoàn toàn** trong từ khác:

```
def dedup_overlap(terms) -> list[str]:
    """Bỏ term bị CHỨA trong term khác cùng danh sách ("sql" ⊂ "sql server"): ..."""
    out = []
    for a in terms:
        if not any(
            a != b and re.search(r"(?<![a-z0-9])" + re.escape(a) + r"(?![a-z0-9])", b)
            for b in terms
        ):
            out.append(a)
    return out
```

Với mỗi từ `a`, hỏi: có từ khác `b` nào mà tìm thấy `a` như một từ nguyên vẹn bên trong `b`? Có → bỏ `a`. ("sql" bị chứa trong "sql server" → chỉ giữ "sql server".) Đo thử trên bộ thi: +1.2 điểm phần trăm precision, mất ~0.8 recall — chấp nhận, vì output sạch hơn cho matching.

5.4 Ghép hai máy lại

```
def extract_skills(text: str, vec, features: list[str], lexicon_set: set, scan_re):
    """Trả về (danh sách skill gộp, top-K tfidf, vị trí xuất hiện đầu của từng skill)."""
    X = vec.transform([text])
    top = [t for t, _ in topk_of(X, 0, features, lexicon_set)]
    ntext = norm(text)
    pos: dict[str, int] = {}
    for m in scan_re.finditer(ntext):
        w = norm(m.group(0))
        if w not in pos and w not in STOP_SKILLS:
            pos[w] = m.start()
    merged = dedup_overlap(list(top) + sorted(set(pos) - set(top)))
    return merged, top, pos
```

- Máy 1 (`topk_of`): 25 kỹ năng điểm cao nhất.
- Máy 2 (`scan_re.finditer`): mọi kỹ năng trong danh bạ xuất hiện trong văn bản, kèm **vị trí** ký tự xuất hiện đầu tiên (dùng ở chương 6 để chia required/preferred).
- `merged`: gộp hai danh sách (phần scan thêm vào chỉ những kỹ năng máy 1 chưa bắt được), lọc trùng chứa một lần cuối. Chữ thường + sắp abc cho ổn định.

Kết quả thực tế: trung bình mỗi CV ra ~25 kỹ năng (chính xác là 25 khi máy 1 đủ), JD ra ~30 kỹ năng bắt buộc + 8 kỹ năng ưu tiên.

Chương 6 — Các trường còn lại của JSON (tên, kinh nghiệm, học vấn, JD)

TF-IDF chỉ lo danh sách kỹ năng. Các trường còn lại là các **regex parser nhỏ** — mỗi trường một quy tắc tìm chữ. Nguyên tắc chung: **không bắt được thì trả `null`, đừng bịa** — null trung thực có giá trị hơn dữ liệu sai.

6.1 CV — `name` (tên ứng viên)

```
def cv_name(text: str) -> str | None:
    """Tên = dòng đầu tiên của CV nếu nó trông giống họ tên người. ..."""
    NAME_STRICT = re.compile(r"^[A-Z][a-zA-Z.'-]+(?: [A-Z][a-zA-Z.'-]+){1,4}$")
    for line in str(text).splitlines()[:1]:
        line = ICON_RE.sub("", clean_line(line)).strip(" -|")
        if not line or "@" in line or any(c.isdigit() for c in line):
            return None
        if NAME_JUNK_RE.search(line) or not NAME_STRICT.match(line):
            return None
        return line
    return None
```

Quy tắc nghiêm ngặt: dòng **đầu tiên** phải là 2–5 từ, mỗi từ viết hoa đầu, chữ cái và dấu `' . -`, không có số, không có @, không khớp danh sách từ rác (NAME_JUNK_RE: education, summary, curriculum vitae, university...).

Sự thật thú vị về corpus: trên 643 CV của bộ matching, quy tắc này bắt được **0 tên**. Không phải máy dở — corpus này sinh ra đã *ẩn tên người* và các dòng chữ bị dính vào nhau ("SummaryAccountant with over a decade...") nên không còn dòng tên riêng. Trả `null` là đúng. Trên bộ test ngoài gồm **CV thật từ PDF** (chương 8), cùng quy tắc bắt được tên ở **101/1000** CV — vì CV thật thường mở đầu bằng tên. Cùng một quy tắc, dữ liệu khác, kết quả khác — đó là lý do phải đo phủ trường trên từng bộ dữ liệu riêng.

6.2 CV — `experience_years` (số năm kinh nghiệm), 2 tầng

Tầng 1 — tìm câu tuyên bố: regex dạng "N years ... experience" (VD: "over 10 years of experience in accounting"), lấy số lớn nhất ≤ 45 (loại "2019 years" do lỗi tách).

Tầng 2 — cộng dồn khoảng thời gian: nếu không có câu tuyên bố, tìm mục "Experience", trong đó gom các cặp năm `2015 – 2018`, `2019 – Present`, **gộp các khoảng chồng nhau** (làm việc 2015–2018 và 2018–2020 là 5 năm chứ không phải 7) rồi cộng lại:

```
spans.sort()
merged, total = list(spans[0]), 0
for a, b in spans[1:]:
    if a <= merged[1]:                      # chồng nhau → kéo dài khoảng đang gộp
        merged[1] = max(merged[1], b)
    else:                                   # rời nhau → cộng khoảng cũ, mở khoảng mới
        total += merged[1] - merged[0]
        merged = [a, b]
total += merged[1] - merged[0]
```

Phủ trên 643 CV: **143 CV (22.2%)** có số năm. Phần còn lại CV viết kiểu "3/2015 – 6/2018 tại công ty A; 2019 – nay tại công ty B" không theo khuôn nào — không ép, trả null.

6.3 CV — `education` (học vấn)

```
def cv_education(text: str) -> dict:
    text = str(text)
    block = text
    msec = SEC_HEADER_RE.search(text)          # chia CV thành mục theo tiêu đề in hoa
    if msec and re.match(r"(?i)(education|academic)", msec.group(1)):
        nxt = SEC_HEADER_RE.search(text, msec.end())
        block = text[msec.end(): nxt.start() if nxt else len(text)][:900]
    ...
```

Cách làm: (1) cắt riêng **mục Education** (từ tiêu đề "EDUCATION" tới tiêu đề mục tiếp theo, tối đa 900 ký tự); (2) trong đó tìm:

- **degree** — bằng cấp, chuẩn hóa về từ điển cố định: PhD / Master / Bachelor / MBA / Associate / Diploma ("B.S.", "Bachelor of Science", "BS" → cùng một dạng);
- **major** — chuyên ngành: mẫu "Bachelor *of* X" hoặc "degree *in* X";
- **university** — tên trường: "X University" hoặc "University of X".

Phủ: degree **99.7%**, major **62.1%**, university **93.0%**. Degree gần như luôn bắt được vì CV nào cũng ghi bằng; major khó hơn vì nhiều CV chỉ ghi tên trường.

6.4 JD — `job_title` (chức danh)

Tầng 1: dòng "Job Title: ..." (cắt trước các từ khóa Location/Duration/Pay — nhiều JD ghi "Job Title: Java Developer Location: New York" liền một dòng). Tầng 2 (fallback — *phương án dự phòng*): dòng đầu tiên trong 5 dòng nếu 1–10 từ và không là rác ("Job details", "Location ...", "Salary ..." — danh sách từ rác tường minh). Phủ: **179/351 (51.0%)** — sạch hơn nhiều so với trước khi lọc rác (56.1% nhưng lắm title rác).

6.5 JD — chia `required_skills` / `preferred_skills`

JD thường có cụm đánh dấu phần "có thì tốt": *nice to have*, *preferred*, *bonus*, *desirable*. Ý tưởng: kỹ năng **xuất hiện lần đầu sau** cụm đó → ưu tiên; trước → bắt buộc. May mắn là ta đã lưu vị trí ký tự của từng kỹ năng ở chương 5.4!

```
def jd_split_skills(text: str, skills: list[str], pos: dict[str, int]) -> tuple[list, list]:
    """Skill có lần xuất hiện ĐẦU TIÊN sau marker preferred → preferred, ngược lại → required."""
    mp = PREFERRED_RE.search(str(text))
    if not mp:
        return list(skills), []
    cut = mp.start()
    required  = [s for s in skills if pos.get(norm(s), 10**9) < cut]
    preferred = [s for s in skills if pos.get(norm(s), 10**9) >= cut]
    return required, preferred
```

`pos.get(norm(s), 10**9)`: kỹ năng mà scan không tìm thấy vị trí (hiếm, chỉ có trong top TF-IDF) coi như vị trí vô cùng → rơi vào preferred (an toàn: không nên bắt buộc thứ không tìm thấy trong văn bản). Không có marker → toàn bộ là required.

6.6 JD — `required_years` (số năm yêu cầu)

Tìm mẫu số năm ("5+ years"), nhưng **chỉ nhận khi trong 60 ký tự tiếp theo** có chữ experience/expertise — để "5 years ago I..." không bị bắt nhầm. Lấy max ≤ 30. Phủ: **234/351 (66.7%)**.

6.7 Hai sản phẩm mẫu (nguyên vẹn)

CV c94f1a9af5c3 — kế toán, học Cincinnati:


```
{
  "doc_id": "c94f1a9af5c3",
  "name": null,
  "skills": ["peachtree", "cpa", "accounting", "business administration", "network", "accountant"],
  "experience_years": null,
  "education": {"degree": "B.S", "major": null, "university": "University of Cincinnati"}
}
```


JD 0e5f1393c8cd — vị trí Dynamics 365:


```
{
  "doc_id": "0e5f1393c8cd",
  "job_title": "Software Engineering Supervisor (Lead Dynamics 365 Developer - ERP) - W2",
  "required_skills": ["dynamics 365", "erp", "software engineering", "supervisor", "developer"],
  "preferred_skills": ["curious", "strategic thinking", "technical knowledge", "articulate"],
  "required_years": 6
}
```


Nhìn `preferred_skills`: "curious", "articulate" — từ mềm mô tả tính cách lọt qua vì danh bạ ngoài (6.108 kỹ năng) lấy cả từ mô tả con người. Đây là **điểm yếu đã biết**, ghi lại trung thực trong README — bước cải thiện kế tiếp có thể lọc soft skill (kỹ năng mềm/mô tả tính cách) ra khỏi danh bạ.

Chương 7 — Chấm điểm máy: precision / recall và bài thi 80/20

7.1 Hai khái niệm chấm điểm

- **Precision (độ chính xác):** "máy ghi ra 25 kỹ năng — bao nhiêu % là đúng?" Cao = máy *ít nói dối*.
- **Recall (độ bao phủ):** "CV thật liệt kê 40 kỹ năng — máy bắt được bao nhiêu %?" Cao = máy *ít bỏ sót*.

Hai bên đánh đổi: chỉ ghi 1 kỹ năng chắc chắn đúng → precision 100% nhưng recall thảm hại; ghi 500 từ bừa → recall tốt nhưng precision tệ. Phải cân bằng.

7.2 Bài thi 80/20 (held-out evaluation)

Nguyên tắc giáo dục: **thi bằng đề chưa từng thấy.** Chia 643 CV: 80% (514 CV) cho máy "học" (dựng danh bạ + bảng idf), 20% (129 CV) làm "đề thi". Nếu dùng chính dữ liệu đã học để chấm điểm, máy chỉ đang *đọc lại bài*, không phải *kiểm tra hiểu biết* (jargon: overfitting — học vẹt).

```
def heldout_eval(cv: pd.DataFrame, extra: list[str]) -> None:
    """Eval 80/20 Y HỆT pipeline cũ ... để số liệu so sánh trực tiếp."""
    rng = np.random.RandomState(RANDOM_STATE)       # seed 42: xáo kiểu gì cũng y hệt lần sau
    idx = rng.permutation(len(cv))                  # xáo trộn thứ tự 643 CV
    n_tr = int(len(cv) * EVAL_SPLIT)                # int(643 * 0.8) = 514
    tr, ho = cv.iloc[idx[:n_tr]], cv.iloc[idx[n_tr:]]   # 514 học / 129 thi

    lex_ho = build_lexicon(tr["text"])              # danh bạ CHƯA từng thấy 129 đề thi
    lex_ho_set = set(lex_ho) | set(extra)           # + danh bạ ngoài (không thấy CV nào)
    vec_ho = fit(tr["text"])                        # bảng IDF cũng chỉ học từ 514 CV
    feat_ho = list(vec_ho.get_feature_names_out())
    X_ho = vec_ho.transform(ho["text"])             # chấm đề thi bằng bảng cũ — KHÔNG fit lại

    precs, recs, n_gt = [], [], 0
    for i, (_, r) in enumerate(ho.iterrows()):
        gt = set(parse_skills_block(r["text"]))     # đáp án: block "Skills" thật của CV
        topk = topk_of(X_ho, i, feat_ho, lex_ho_set)  # 25 kỹ năng máy chọn
        if not topk or not gt:
            continue                                # không đáp án → không chấm được, bỏ qua
        hits = len({t for t, _ in topk} & gt)       # & = giao tập: đếm trùng nhau
        precs.append(hits / len(topk))
        recs.append(hits / len(gt))
        n_gt += 1
    print(f"  precision@{TOP_K} = {np.mean(precs):.2%}")
    print(f"  recall@{TOP_K}    = {np.mean(recs):.2%}")
```

Bốn chi tiết đáng ghi nhớ:

1. `RandomState(42)` — *seed* (hạt giống ngẫu nhiên cố định). "Ngẫu nhiên" nhưng tái lập được: chạy 100 lần vẫn xáo y như vậy → số 39.75% là con số ổn định, không phải may rủi.
2. `fit(tr["text"])` **chỉ trên 514 CV**, còn `transform(ho["text"])` cho 129 đề thi — dùng bảng idf cũ để chấm. (Từng mắc lỗi thật: sơ suất fit lên cả bộ → precision rơi từ 39.75% xuống 5.00%. Con số đẹp bất thường → kiểm tra pipeline trước, tin số sau.)

3. `gt` = *ground truth* (đáp án chuẩn) — tách bằng chính `parse_skills_block` (chương 3.3), giữ nguyên không chuẩn hóa để so sánh công bằng với số liệu cũ của pipeline trước.
4. CV không có block Skills (không có đáp án) → bỏ qua: chấm được **105/129** CV.

7.3 Đọc kết quả: 39.75% / 25.33% — thấp mà dùng được

```
precision@25 = 39.75%   (trong 25 kỹ năng máy chọn, ~10 cái đúng đáp án)
recall@25    = 25.33%   (đáp án trung bình ~16 kỹ năng, máy bắt ~4)
```

Vì sao thấp mà vẫn chấp nhận? Vì **đáp án hẹp và vòng tròn (circular)**: ground truth là block "Skills" mà CV viết, nhưng block đó mỗi CV một kiểu viết, còn danh bạ 6.815 kỹ năng thì rộng. Nhiều kỹ năng máy chọn *đúng thật* (như "accounting") nhưng đáp án CV ghi dạng khác ("Accounting/Bookkeeping") → bị tính sai. Chỉ số này là **baseline** (mốc chuẩn) để so sánh khi thêm phương pháp khác (BERT) — quan trọng là biết con số đến từ đâu, không phải cứ số cao là tốt.

7.4 Các thí nghiệm đã làm (và bài học)

Thí nghiệm	P@25	R@25	Kết luận
Baseline (danh bạ 707, K=25)	47.53%	28.44%	mốc so sánh cũ
+ danh bạ ngoài 6.108	39.75%	25.33%	P giảm là **đáng chờ đợi**: đáp án hẹp đếm kỹ năng mới là "sai", nhưng coverage ngoài tăng ~2.3× — cái matching cần
Lấy K=10	50.29%	14.58%	P cao nhưng sót nhiều
Lấy K=15	49.03%	20.75%	
K=25 (chọn)	46.37%	29.25%*	cân bằng tốt nhất cho matching
Lấy K=30	45.88%	31.46%	P giảm nhanh hơn R tăng

* dòng K=25 trong bảng sweep đo trên cấu hình danh bạ 707; số chính thức của pipeline hiện tại (danh bạ 6.815) là 39.75/25.33.

Bài học lớn nhất: **mỗi cải tiến phải đo trên cùng một mặt đo** — nếu đổi cách chấm cùng lúc đổi phương pháp thì không biết cái gì làm số tăng/giảm.

Chương 8 — Thi ngoài: bộ test 1.099 file từ nguồn khác

8.1 Vì sao cần "thi trường cùng"

Học và thi trên cùng một nguồn dữ liệu (cùng kiểu CV sinh ra máy) vẫn có rủi ro *quen*. Số học sinh giỏi một trường đi thi liên trường mới biết trình độ thật. Nên ta dựng bộ test từ **nguồn khác hoàn toàn**:

- **1.000 CV thật từ PDF** (nguồn d4rk3r trên HuggingFace) + 71 CV Kaggle bù đắp — đọc text bằng `pypdf`, giữ file ≥500 ký tự, loại trùng;
- **100 JD** từ bộ job-descriptions.

Kiểm tra trùng với corpus chính: CV 1/1000, JD 0/100 — sạch.

8.2 Kết quả phủ trường

Bộ	≥1 skill	≥5	≥10	TB skill/doc	zero
999 CV ngoài	94.3%	74.2%	57.3%	25.2	57
100 JD ngoài	100%	99.0%	93.0%	35.9	0

- 94.3% CV ngoài có ít nhất 1 kỹ năng → pipeline **dùng được với dữ liệu thật**, không chỉ khớp với corpus sinh ra máy.
- **57 CV "mù"** — điều tra kỹ thì đó là file PDF bị hỏng khi đọc: trung bình chỉ ~1.237 ký tự (bình thường ~3.823), chữ tiếng Việt mất dấu, ký tự lạ `\uf0b1` (biểu tượng riêng của font trong PDF, không phải chữ). Không phải lỗi thuật toán — là lỗi *đầu vào*; ghi rõ để không ai hiểu lầm.

Chương 9 — Sơ đồ tổng thể + bàn giao cho bạn cùng nhóm

9.1 Luồng dữ liệu đầy đủ

```
HuggingFace (cần mạng)                    script                    sản phẩm
──────────────────────────────────────────────────────────────────────────────

matching_train/test.csv  ──▶ download_data.py ─▶ data/csv/*.csv
        │
        ▼
build_pairs.py   ──▶ data/csv/pairs.csv (93.733 cặp, KHÔNG có text)
                     data/csv/cv_docs.csv (643) + jd_docs.csv (351)
        │
        ▼
build_lexicon_extra.py ──▶ data/skills_lexicon_extra.txt (6.108)
        │
        ▼
build_testset.py ──▶ data/csv/test_cv_docs.csv (1000) + test_jd_docs.csv (100)
        │
        ▼
extract_json.py  ──▶ data/extracted/cv/<doc_id>.json      (643 file)
                     data/extracted/jd/<doc_id>.json      (351 file)
                     data/extracted/test_cv/, test_jd/    (1.099 file)
                     + in: danh bạ 6.815, vocab 32.704, P/R 39.75/25.33
```

9.2 Bạn nhóm dùng output thế nào? 10 dòng code

```
import pandas as pd, json

pairs = pd.read_csv("data/csv/pairs.csv")   # pair_id, resume_id, jd_id, label, split, ...

def load(bo, doc_id):                        # bo = "cv" hoặc "jd"
    with open(f"data/extracted/{bo}/{doc_id}.json", encoding="utf-8") as f:
        return json.load(f)

row = pairs.iloc[0]
cv = load("cv", row["resume_id"])
jd = load("jd", row["jd_id"])

can   = set(jd["required_skills"])           # JD cần gì
co    = set(cv["skills"])                    # CV có gì
trung = can & co                             # giao tập: cần mà có
thieu = can - co                             # hiệu tập: cần mà không có
print(row["label"], f"đủ {len(trung)}/{len(can)} skill bắt buộc; thiếu:", sorted(thieu)[:5])
```

Chạy với từng dòng của pairs.csv → mỗi cặp có một "khoảng cách kỹ năng" → đó là đặc trưng (feature) đầu vào cho model matching của bạn nhóm. Nhãn `label` + cột `split` cho biết cặp nào thuộc tập học / tập thi.

9.3 Chạy lại toàn bộ (theo đúng thứ tự)

```
python data/download_data.py        # 1. tải dữ liệu gốc (cần mạng)
python data/build_pairs.py          # 2. làm sạch, dedup, PII → data/csv/
python data/build_lexicon_extra.py  # 3. gom 6.108 skill ngoài (cần mạng)
python data/build_testset.py        # 4. dựng bộ test ngoài (cần mạng)
python data/extract_json.py         # 5. trích xuất JSON + chấm điểm held-out
```

Chạy riêng bước 5 (khi CSV đã có) là đủ để tái tạo toàn bộ output JSON. Muốn sửa danh bạ: mở `data/skills_lexicon.txt` sửa từng dòng (mỗi dòng một kỹ năng).

Chương 10 — Bảng thuật ngữ

Thuật ngữ	Nghĩa đơn giản
TF-IDF	Công thức chấm điểm từ: quan trọng = hay xuất hiện trong văn bản này × hiếm ở nơi khác
TF	Tần suất từ trong chính văn bản này (đã làm dịu logarit)
IDF	Độ hiếm của từ trên toàn bộ tập văn bản — càng hiếm càng giá trị
logarit (ln)	Hàm làm số lớn "xẹp" lại: ln(10)≈2.3, ln(100)≈4.6
L2 chuẩn hóa	Chia vector cho độ dài của chính nó → độ dài bằng 1, so sánh công bằng
tokenizer	Bộ tách chữ thành từ ("B.S. 2019" → ["b.s.", "2019"])
bigram / n-gram	Cụm 2 (hay n) từ liền nhau được coi như một mục từ: "sql server"
min_df	Ngưỡng lọc: từ phải xuất hiện trong ≥N văn bản mới được vào từ vựng
stop words	Từ vô nghĩa bị loại sẵn: the, and, with...
danh bạ (lexicon)	Danh sách các từ được công nhận là kỹ năng
regex	Biểu thức chính quy — công thức mô tả mẫu chữ để tìm/thay thế
lookaround	Mảnh regex kiểm tra "đằng trước/sau KHÔNG phải..." mà không lấy ký tự đó
chuỗi con (substring)	Dãy ký tự nằm trong dãy khác; "cat" ⊂ "category"
md5 / hash	Hàm băm: văn bản → dãy ký tự nhận dạng duy nhất (12 ký tự đầu)
PII	Thông tin cá nhân (email, điện thoại) — được che thành [EMAIL]/[PHONE]/[URL]
dedup	Loại trùng lặp, giữ mỗi nội dung một bản
ma trận thưa (sparse)	Bảng số mà đa số ô là 0; máy chỉ lưu ô khác 0 cho nhẹ
ground truth (GT)	Đáp án chuẩn dùng để chấm — ở đây là block "Skills" CV thật viết
held-out	Phần dữ liệu giấu không cho học, chỉ dùng để thi
seed (RandomState 42)	Hạt giống số ngẫu nhiên cố định → xáo trộn tái lập được
precision	Trong những gì máy ghi ra, bao nhiêu % đúng
recall	Trong tổng số điều cần bắt, máy bắt được bao nhiêu %
giao / hiệu tập (`&` / `-`)	Phần chung / phần dư thừa giữa hai danh sách
overfitting	Học vẹt: ghi nhớ dữ liệu cũ, làm tệ trên dữ liệu mới
baseline	Mốc chuẩn để so sánh các cải tiến về sau
JSON	Định dạng dữ liệu có cấu trúc {khóa: giá trị}, máy lẫn người đều đọc được
null	Giá trị "không có" — trung thực hơn việc bịa số 0

Chương 11 — Câu hỏi ôn tập

Câu 1. Vì sao "peachtree" (2 lần) được điểm cao hơn "accounting" (7 lần)?

> *Đáp án:* điểm = TF×IDF. "accounting" xuất hiện trong 310/994 văn bản → idf thấp (2.16); "peachtree" chỉ 15/994 → idf cao (5.13). Độ hiếm bù quá mức đếm: 1.69×5.13 > 2.95×2.16. Bài học: TF-IDF đo **đặc trưng**, không đo **số lượng**.

Câu 2. `topk_of` dừng ở 25 skill và lọc theo danh bạ — nhưng vẫn cần máy dò scan thứ hai là vì 3 lý do nào?

> *Đáp án:* (1) TF-IDF chỉ nhận từ nguyên vẹn — "experienced" không sinh "experience"; (2) `min_df=3` loại từ vựng các kỹ năng hiếm (xuất hiện <3 văn bản); (3) kỹ năng có trong CV nhưng không lọt top 200 điểm sẽ bị máy 1 bỏ sót. Scan quét toàn văn theo danh bạ nên bắt được cả 3 trường hợp.

Câu 3. Regex scan dùng `(?<![a-z0-9])...(?![a-z0-9])` thay vì `\b`. `\b` hỏng với kỹ năng nào, vì sao?

> *Đáp án:* kỹ năng chứa ký tự đặc biệt, ví dụ "c++": sau `+` là `,` — cả hai đều không phải "ký tự từ" nên không tồn tại biên từ nào ở đó → `\bc\+\+\b` không khớp bao giờ. Lookaround chỉ yêu cầu "kế bên không phải chữ/số" nên dấu phẩy được thông qua.

Câu 4. Trong `heldout_eval`, vì sao `fit` chỉ chạy trên 514 CV mà `transform` chạy trên 129 CV còn lại? Nếu sơ suất fit lên cả 643 thì hậu quả gì?

> *Đáp án:* đề thi phải chưa từng thấy: danh bạ và bảng idf dựng từ 514 CV học, rồi *dùng nguyên bảng đó* chấm 129 đề thi. Fit nhầm lên cả bộ → từ kỹ năng của đề thi có mặt trong danh bạ/idf từ đầu → máy "nhìn đề trước" (overfitting), precision rơi từ 39.75% về 5.00% — lỗi đã gặp thật trong quá trình làm.

Câu 5. CV ngoài bộ test: 94.3% có ≥1 kỹ năng nhưng 57 CV = 0. Điều tra ra nguyên nhân gì, và đó là lỗi của thuật toán hay dữ liệu?

> *Đáp án:* 57 CV đó là file PDF hỏng khi trích text: trung bình ~1.237 ký tự (bình thường ~3.823), mất dấu tiếng Việt, ký tự lạ `\uf0b1` của font nhúng. Lỗi **đầu vào**, không phải thuật toán — pipeline được thiết kế in rõ trường hợp zero để không giấu lỗi.

Hết tài liệu. Số liệu trong bài tái lập bằng `python data/extract_json.py` từ thư mục gốc project; chi tiết từng script trong `data/README.md`.